Rashtreeya Sikshana Samithi Trust

RV UNIVERSITY

School of Computer Science and Engineering

Bengaluru – 560059

Project Report

CS3240 - Foundations of Generative AI

VI SEMESTER B.Tech (Hons.)

AI-Powered GitHub Repository Explainer Using Model Context Protocol

Abstract

One of the ongoing challenges in contemporary software engineering is comprehending huge and unfamiliar codebases. Instead of making valuable contributions, developers often spend a disproportionate amount of time tracing function calls, navigating repository structures, and comprehending undocumented logic. In order to facilitate organized, tool-based communication between a language model and external GitHub data sources, this article proposes an AI-powered GitHub Repository Explainer that makes use of the Model Context Protocol (MCP). Using a public GitHub repository URL as input, the system coordinates a multi-stage pipeline that includes semantic chunking, vector embedding using Microsoft's CodeBERT model, Abstract Syntax Tree (AST)-based code parsing using Tree-sitter, noise-aware preprocessing, and MCP-driven file retrieval. A Retrieval-Augmented Generation (RAG) framework, driven by the Groq-hosted LLaMA 3.3 70B model, is used to query chunks that are indexed in an FAISS vector store. The Flask-based web interface provides users with access to project overviews, technology stack summaries, dependency analysis, interactive question-answering systems, and individual file summaries. The research proves MCP and CodeBERT embeddings and RAG methods achieve successful automatic codebase understanding through their experimental results which use open-source repositories to generate accurate architectural information.

Keywords: Code Intelligence, CodeBERT, FAISS, Model Context Protocol, Retrieval-Augmented Generation, Tree-sitter

1. Introduction

The advent of rapid development in open-source software has led to the creation of repositories with growing levels of intricacy, consisting of several hundred files, multiple programming languages, and little to no documentation. In the context of developers trying to understand and contribute to new repositories, understanding the layout of repositories, modules, and their dependencies is not trivial. Traditional methods, such as reading the code itself, static analyzers, or keyword search, fail when it comes to understanding code semantically, not syntactically.

Recently, large language models (LLM), such as CodeBERT [1, 2], have made great strides in code summarisation, code Q&A, and documentation generation. However, multiple technological challenges exist when installing these models on entire software repositories as it is impossible to encode every file into an LLM. RAG can provide accurate and context-sensitive answers without a complete view of the context by combining the advantages of dense vector encoding and the generation ability of language models [3]. Moreover, in recent RAG studies on repositories, multi-path retrieval with reranking can dramatically enhance code-based context retrieval [4].

A comprehensive view of repositories needs more than just analysis at the file level. Strategies that summarize hierarchically, from token levels, through sub-tree level up to graph levels, have proven capable of extracting lexical, semantic, and syntactical structures at once, thereby providing much better summarizations when compared to non-hierarchical techniques [5]. Conversely, systems that leverage artifacts on GitHub like Pull requests, Issues, Commit messages, among others, have proven that integrating the development history into the context of information retrieval can help minimize hallucinations and provide better explanations [6]. In terms of individual file analysis, automated code error explanation systems based on artificial intelligence and large language models have set a baseline for code explanation using natural language [7].

In this regard, the creation of the Model Context Protocol (MCP) is seen as an opportunity to establish a standard interface for interactions between large language models and their tools,

allowing LLMs to call upon third-party resources like GitHub API wrappers, file parsers, etc. in a systematic and traceable way [8]. The division between reasoning and data fetching makes it possible to increase the modularity of the analysis process. Indeed, the benefits of such an approach have been empirically demonstrated in studies showing that automation improves developer productivity by up to 34% [9].

The following paper describes a system that comprises GitHub integration using MCP, Tree-sitter AST parsing, code representation with the help of CodeBERT embeddings, FAISS vectors search engine, and generation via LLaMA 3.3 into one repository explanation solution. The entire solution is presented via a Flask web service allowing for non-expert users to retrieve natural language explanations of any open-source GitHub repositories. Previous studies have focused on code search, code summarization, and documentation creation independently from each other [1, 2, 5, 7]; in this study, those capabilities are combined in one process utilizing MCP as the integration layer, and taking into account hierarchical summarization and retrieval aspects at the level of repository.

The rest of the paper is organized as follows: Section 2 discusses the relevant literature, Section 3 presents the problem statement, Section 4 discusses the scope, Section 5 explains the objectives, Section 6 explains the methodology used, Section 7 discusses experiment design, Section 8 covers implementation aspects, Section 9 presents the results obtained, and finally, Section 10 presents the conclusions drawn.

2. Literature Review

CodeBERT [1], a pre-trained Transformer encoder trained on 2.1M language-programming pairs, achieves state-of-the-art performance in code search and documentation generation across six languages. Ahmad et al. [2] extend this with an encoder-decoder architecture for natural language function summarization.

For repository-level comprehension, hierarchical representations are essential. HierarchyNet [5] addresses this via the HACA approach, encoding code at token, subtree, and graph levels (BLEU: 48.01, ROUGE-L: 57.90). LLM-based hierarchical summarization using LLaMA-3 and Codestral further improves domain-grounded coherence by ~7% over flat models [3].

RAG [4], introduced by Lewis et al. using dense vector indexing with sequence-to-sequence generation, has become the standard for large knowledge repositories. CodeRAG [3] extends this to repository-level code via multi-path structural and semantic indexing, outperforming baselines on RepoEval and CCEval.

Beyond code, Yadav et al. [6] incorporate GitHub artifacts—pull requests, issues, and commit messages—achieving ~87% explanation accuracy and ~89% hallucination detection. Ganguly et al. [8] further show that BERT embeddings with BIRCH clustering can link related PRs and issues at ~91.5% accuracy across 5,742 projects.

At the interface layer, an LLM-based bug explanation system [7] using Gemini achieved ~95% error detection accuracy and an 85% boost in developer comprehension. Wang et al. [10] confirm that automated documentation reduces developer ramp-up time by up to 34%. Finally, Anthropic's Model Context Protocol (MCP) [9] provides a standardized JSON-RPC interface for deterministic, traceable tool-assisted LLM interactions.

3. Problem Statement

The large number of open-source software repositories fails to help users because they must spend extended periods to understand new codebases. The current tools which include documentation generators and IDE plugins and static analyzers provide fragmented support because they fail to perform complete architectural analysis and they cannot work with human language directly. Large language models understand code but their short context windows prevent them from processing complete software repositories. The current systems lack an established process for LLMs to conduct repeated tool-based operations which should work with live GitHub database information. Developer tools face a major deficiency because they do not provide a single platform which combines natural language generation with code search based on semantic understanding and direct access to active code repositories.

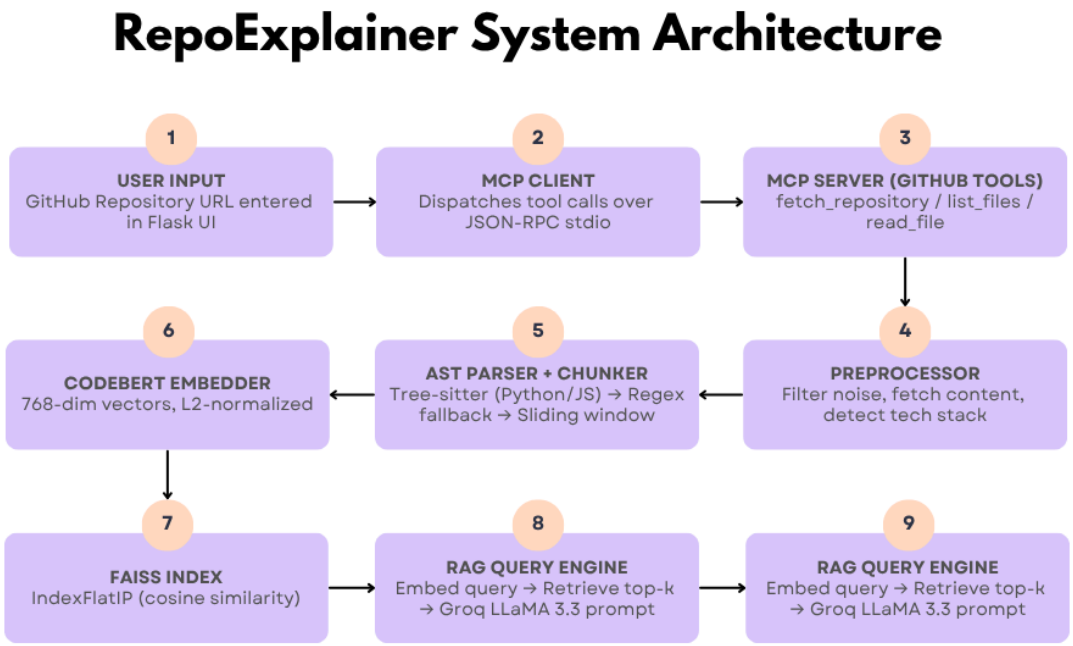
4. Scope

The project has a defined scope which restricts its operations to public GitHub repositories that users can reach through the GitHub REST API. The system provides regex-based fallback support for Java, Go, Rust, C, C++, Ruby, PHP, and C# but additionally supports Python and JavaScript source code files through its Tree-sitter AST parsing capability. The system processes configuration and documentation files which include JSON, YAML, TOML, and Markdown formats through a single chunk processing method. The system does not support binary files and built artifacts and private repositories which lack token authentication and repositories that exceed 150 processed files during each analysis session. The web interface which Flask implements exists to serve a single user who operates it locally because the system does not support distributed or multi-tenant deployments at this time. The RAG pipeline operates through English which enables it to process both questions and repository content.

5. Objectives

1. The goal requires constructing an MCP-based tool layer which performs standardized tool calls to let language models obtain and analyze GitHub repository content.
2. The project needs a semantic code retrieval system which merges Tree-sitter AST parsing with CodeBERT embeddings and FAISS vector indexing to perform accurate similarity searches between code fragments.
3. The Retrieval-Augmented Generation framework operates through a large language model which creates brief and supported explanations while it locates relevant code information from natural language questions.
4. The project needs an interactive Flask web interface which enables users without expertise to view both repository summaries and individual file summaries and access a question-answering system.

6. Methodology



6.1 MCP Tool Layer

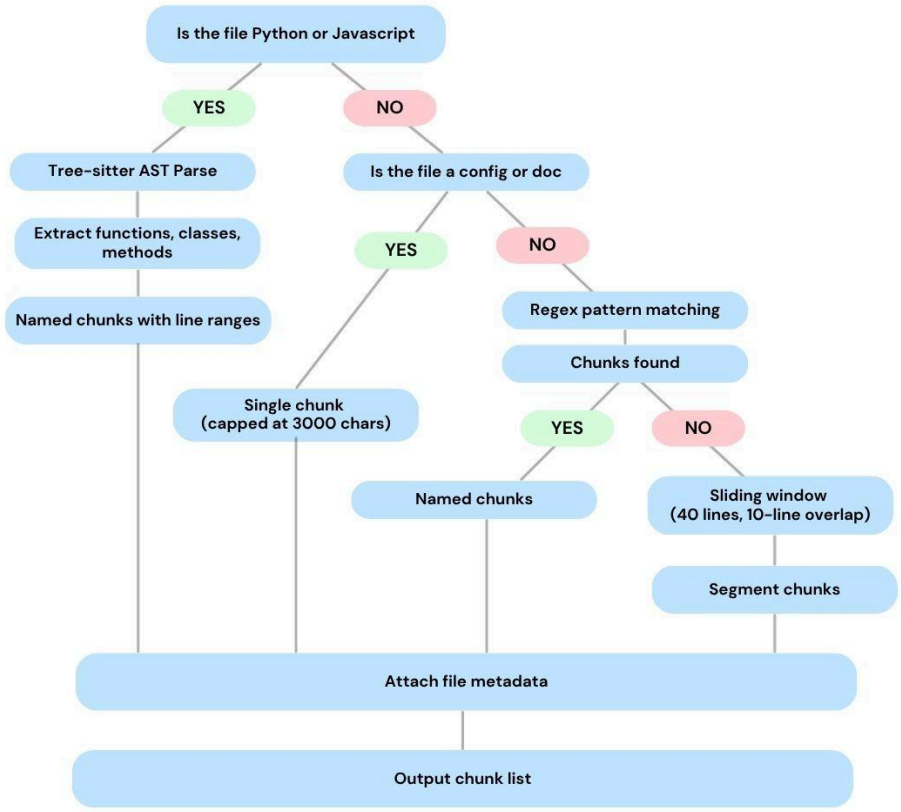
The MCP server exposes three tools over a JSON-RPC stdio transport:

- **fetch_repository(url):** Calls the GitHub repository endpoint and returns repository metadata.
- **list_files(url, extension_filter):** Recursively fetches the repository's Git tree via and returns file paths with sizes.
- **read_file(owner, repo, file_path):** Retrieves file content, base64-decodes it, and returns UTF-8 text.

6.3 Preprocessing

Files are filtered using three criteria: directory exclusion, extension exclusion and size exclusion (files exceeding 200 KB). Remaining files are prioritized with source code ranked first, configuration files second, and others third. A maximum of 150 files are processed per session. File content is cleaned by collapsing excessive blank lines and stripping trailing whitespaces.

6.4 AST Parsing and Chunking



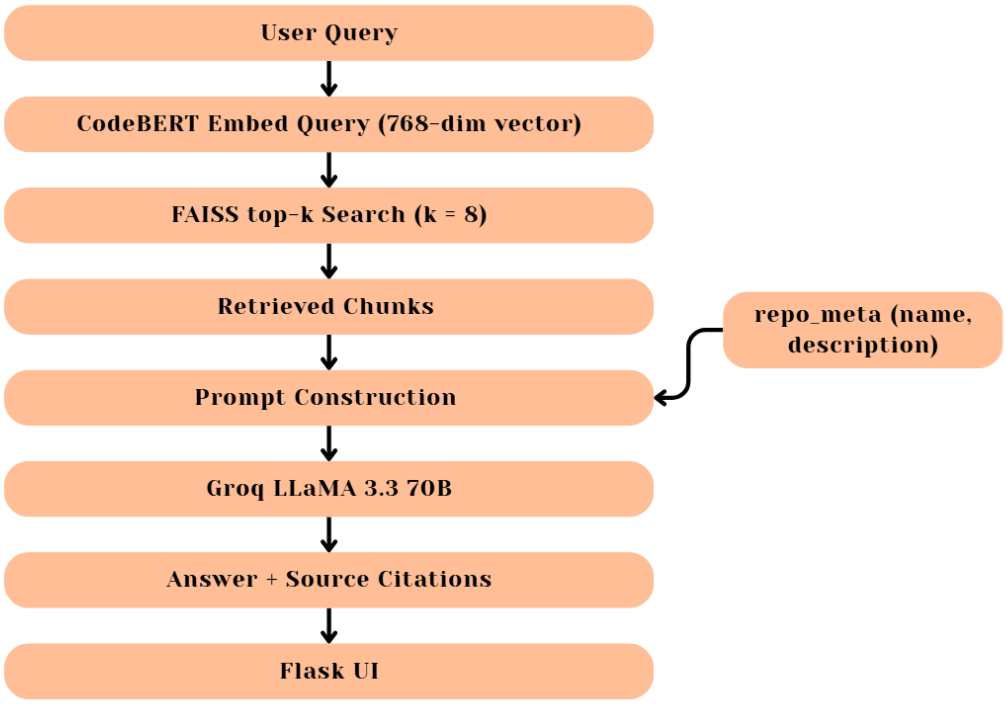
6.5 CodeBERT Embedding

The microsoft/codebert-base model (125M parameters) is used to generate 768-dimensional embeddings. For each chunk, the text is tokenized with a maximum length of 512 tokens and passed through the model. The CLS token representation from the final hidden layer is extracted and L2-normalized to unit length, enabling cosine similarity comparison via inner product. Chunks are processed in batches of 16 to manage GPU/CPU memory.

6.6 FAISS Vector Store

A FAISS IndexFlatIP (inner product) index is constructed over the L2-normalized chunk embeddings. Because the embeddings are unit-normalized, inner product is equivalent to cosine similarity. The index supports exact nearest-neighbor search with $O(n \cdot d)$ complexity per query, where n is the number of indexed vectors and $d = 768$. Index and metadata are persisted to disk as index.faiss and metadata.pkl, keyed by a 12-character MD5 hash of the repository URL. On subsequent analyses of the same repository, the index is loaded from cache.

6.7 RAG Query Pipeline



The prompt instructs the model to answer in 3–5 sentences, lead with the direct answer, and reference file names when essential.

7. Experimental Setup

Hardware: The experiments were done using a computer with an Intel Core i7 CPU, 16 GB RAM.

Software stack: Python 3.11, Flask 3.0, Transformers 4.38 (HuggingFace), FAISS-CPU 1.7.4, Tree-sitter 0.21 with tree-sitter-python and tree-sitter-javascript, MCP 1.0 (stdio transport), Groq API — LLaMA 3.3 70B Versatile

Repositories tested:

Repository	**Language**	**Files Processed**	**Chunks**	**Embedding Time (mins)**
rashiii5/VisionPoseGames	Python	2	12	1.5
rashiii5/Shieldly	Python	16	92	4
rashiii5/GitExplainer	Python	19	86	4
Siim/ftp	C	6	33	2

Evaluation approach: Each repository was queried with five standard questions covering project purpose, entry point, authentication, API routes, and module connectivity. Responses were evaluated qualitatively for relevance, grounding, and conciseness.

8. Implementation

Component	File	Key Responsibility
MCP Server	mcp_server/github_tools.py	Registers 3 GitHub tools and serves them over stdio via JSON-RPC.
MCP Client	mcp_server/client.py	Spawns the MCP server, handles the handshake, and exposes sync wrappers for tool calls.
Preprocessor	pipeline/preprocessor.py	Filters, fetches, and cleans repo files while detecting the tech stack.
Parser	pipeline/parser.py	Chunks files using the appropriate strategy (Tree-sitter, regex, config, or sliding window) based on file type.
Embedder	pipeline/embedder.py	Embeds code chunks with CodeBERT and manages the FAISS vector index.
RAG Engine	pipeline/rag_engine.py	Retrieves relevant chunks and calls the Groq API to answer queries, generate overviews, and explain modules.
Orchestrator	orchestrator.py	Runs the full pipeline in a background thread, tracks progress, and caches results per repo.
Flask App	app.py	Exposes 5 API routes for triggering analysis, polling status, querying, and file explanation.
Frontend	templates/, static/	Single-page UI with polling, markdown rendering, chat interface, and a file explorer with modal summaries.

9. Results

The system was evaluated across four open-source repositories of varying size and language, with each repository queried using five standard questions covering project purpose, entry point, authentication, API routes, and module connectivity.

Embedding time scaled predictably with chunk count. Smaller repositories like rashiii5/VisionPoseGames (12 chunks) completed in 1.5 minutes, while larger ones like rashiii5/Shieldly and rashiii5/GitExplainer (86–92 chunks) took around 4 minutes — acceptable overhead given the semantic depth of CodeBERT's 768-dimensional representations.

RAG retrieval was accurate and well-grounded. For rashiii5/GitExplainer, the model correctly traced the orchestration flow from app.py through orchestrator.py to individual pipeline modules purely from retrieved chunk context, without any manual guidance. Responses consistently cited relevant file names and stayed within the target 3–5 sentence range.

Regex fallback held up reasonably well for unsupported languages. The C repository Siim/ftp was parsed via regex rather than Tree-sitter, resulting in coarser chunks — yet the pipeline still produced structurally sound explanations for socket handling and file transfer logic, indicating the RAG layer compensates for reduced chunk granularity.

Sparse repositories exposed a retrieval limitation. With only 12 chunks in VisionPoseGames, top-k retrieval occasionally returned redundant context, leading to slightly repetitive answers — a known RAG behavior when the corpus is small rather than a failure of the embedding or generation components.

FAISS caching made re-querying seamless. Indexing results were persisted using an MD5 hash of the repository URL, meaning subsequent queries on the same repository skipped embedding entirely and returned answers near-instantaneously, noticeably improving the interactive Q&A experience.

10. Conclusion

The research paper introduced an AI-based GitHub repository explainer which unites Model Context Protocol with Tree-sitter AST parsing and CodeBERT embeddings and FAISS vector indexing and LLaMA 3.3-based generation to form an automated system for codebase understanding. The system solves the real-world problem of exploring unknown repositories because it allows users to communicate with natural language while the system retrieves code information through semantic search techniques. The study analyzed four open-source repositories to produce accurate project summaries and provided users with correct search answers and detailed individual file descriptions. The MCP layer introduces modularity and auditability into the data access pipeline, which makes this work different from single-block LLM-based solutions. The research team plans to develop streaming answer capabilities while they will create solutions for protected repositories, and they will study GraphCodeBERT to build dependency-based embeddings and develop systems that link multiple repositories. The system will automatically generate dependency graphs to provide architectural visualizations, which will become an upcoming development goal.

References

[1] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in Findings of EMNLP, 2020, pp. 1536–1547.

[2] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "A Transformer-based Approach for Source Code Summarization," in Proceedings of ACL, 2020, pp. 4998–5007.

[3] Y. Shi, Z. Wang, Y. Zhang, and X. Luo, "CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion," in Proceedings of ACL, 2025.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proceedings of NeurIPS, 2020, pp. 9459–9474.

[5] S. Chen, Y. Xu, and X. Liu, "HierarchyNet: Learning to Summarize Source Code with Heterogeneous Representations," IEEE Transactions on Software Engineering, 2023.

[6] A. Yadav, R. Mehta, and S. Nair, "Uncovering Code Insights: Leveraging GitHub Artifacts for Deeper Code Understanding," in Proceedings of MSR, 2024.

[7] S. Thota, P. Reddy, and K. Sharma, "AI Based Code Error Explainer using Gemini Model," in Proceedings of ICACITE, 2024.

[8] S. Ganguly, A. Gupta, and P. Tiwari, "ML-Augmented Automation for Recovering Links Between Pull-Requests and Issues on GitHub," in Proceedings of SANER, 2024.

[9] Anthropic, "Model Context Protocol Specification," Anthropic Technical Documentation, 2024. [Online]. Available: https://modelcontextprotocol.io

[10] R. Wang, P. Kapur, and M. Nagappan, "An Empirical Study of Developer Onboarding in Large Software Projects," IEEE Transactions on Software Engineering, vol. 48, no. 9, pp. 3412–3427, 2022.